MK Praktikum Unggulan Universitas Gunadarma

Nama Mata Kuliah: Praktikum Komputasi Big Data (Tingkat 2)

Pertemuan VII

Pada pertemuan ini Anda akan melanjutkan tahapan pengolahan data yaitu data training dan model generation.

Import Dataset

In [1]:
```
# Import library
import pandas as pd
import numpy as np

# Import dataset
data = pd.read_csv('https://gitlab.com/andreass.bayu/file-directory/-/raw

# Lihat 5 data teratas
data.head(5)
```

Out[1]:

	age	workclass	fnlwgt	education	educational-num	marital-status	occupation	relationship
0	25	Private	226802	11th	7	Never-married	Machine-op-inspct	Own-child
1	38	Private	89814	HS-grad	9	Married-civ-spouse	Farming-fishing	Husband
2	28	Local-gov	336951	Assoc-acdm	12	Married-civ-spouse	Protective-serv	Husband
3	44	Private	160323	Some-college	10	Married-civ-spouse	Machine-op-inspct	Husband
4	18	?	103497	Some-college	10	Never-married	?	Own-child

Review Dataset

In [2]:
```
# Lihat statistik deskripsi data dari tiap kolom dengan menggunakan fungs
data.describe()
```

Out[2]:

	age	fnlwgt	educational-num	capital-gain	capital-loss	48
count	48842.000000	4.884200e+04	48842.000000	48842.000000	48842.000000	
mean	38.643585	1.896641e+05	10.078089	1079.067626	87.502314	
std	13.710510	1.056040e+05	2.570973	7452.019058	403.004552	
min	17.000000	1.228500e+04	1.000000	0.000000	0.000000	
25%	28.000000	1.175505e+05	9.000000	0.000000	0.000000	
50%	37.000000	1.781445e+05	10.000000	0.000000	0.000000	
75%	48.000000	2.376420e+05	12.000000	0.000000	0.000000	
max	90.000000	1.490400e+06	16.000000	99999.000000	4356.000000	

In [3]:
```
# Lihat tipe data dari tiap kolom dengan menggunakan fungsi dtypes
data.dtypes
```

Out[3]:
```
age               int64
workclass           str
fnlwgt            int64
education           str
educational-num   int64
marital-status      str
occupation          str
relationship        str
race                str
gender              str
capital-gain      int64
capital-loss      int64
hours-per-week    int64
native-country      str
income              str
dtype: object
```

In [4]:
```
# Melihat jumlah atribut dan dimensi data (baris dan kolom) dengan menggu
data.shape
```

Out[4]:
```
(48842, 15)
```

In [5]:
```
# Menghitung dan melihat jumlah data per label kelas
for col in data.columns:
  if pd.api.types.is_string_dtype(data[col]):
    print('Attribute name:',col)
    print('--------------------')
    print(data[col].value_counts())
    print('--------------------')
```

```
Attribute name: workclass
-------------------
workclass
Private             33906
Self-emp-not-inc     3862
Local-gov            3136
?                    2799
State-gov            1981
Self-emp-inc         1695
Federal-gov          1432
Without-pay            21
Never-worked           10
Name: count, dtype: int64
-------------------
Attribute name: education
-------------------
education
HS-grad         15784
Some-college    10878
Bachelors        8025
Masters          2657
Assoc-voc        2061
11th             1812
Assoc-acdm       1601
10th             1389
7th-8th           955
Prof-school       834
9th               756
12th              657
Doctorate         594
5th-6th           509
1st-4th           247
Preschool          83
Name: count, dtype: int64
-------------------
Attribute name: marital-status
-------------------
marital-status
Married-civ-spouse       22379
Never-married            16117
Divorced                  6633
Separated                 1530
Widowed                   1518
Married-spouse-absent      628
Married-AF-spouse           37
Name: count, dtype: int64
-------------------
Attribute name: occupation
-------------------
occupation
Prof-specialty       6172
Craft-repair         6112
Exec-managerial      6086
Adm-clerical         5611
Sales                5504
Other-service        4923
Machine-op-inspct    3022
?                    2809
Transport-moving     2355
Handlers-cleaners    2072
```

```
Farming-fishing      1490
Tech-support         1446
Protective-serv       983
Priv-house-serv       242
Armed-Forces           15
Name: count, dtype: int64
-------------------
Attribute name: relationship
-------------------
relationship
Husband           19716
Not-in-family     12583
Own-child          7581
Unmarried          5125
Wife               2331
Other-relative     1506
Name: count, dtype: int64
-------------------
Attribute name: race
-------------------
race
White                 41762
Black                  4685
Asian-Pac-Islander     1519
Amer-Indian-Eskimo      470
Other                   406
Name: count, dtype: int64
-------------------
Attribute name: gender
-------------------
gender
Male      32650
Female    16192
Name: count, dtype: int64
-------------------
Attribute name: native-country
-------------------
native-country
United-States                43832
Mexico                         951
?                              857
Philippines                    295
Germany                        206
Puerto-Rico                    184
Canada                         182
El-Salvador                    155
India                          151
Cuba                           138
England                        127
China                          122
South                          115
Jamaica                        106
Italy                          105
Dominican-Republic             103
Japan                           92
Guatemala                       88
Poland                          87
Vietnam                         86
Columbia                        85
Haiti                           75
```

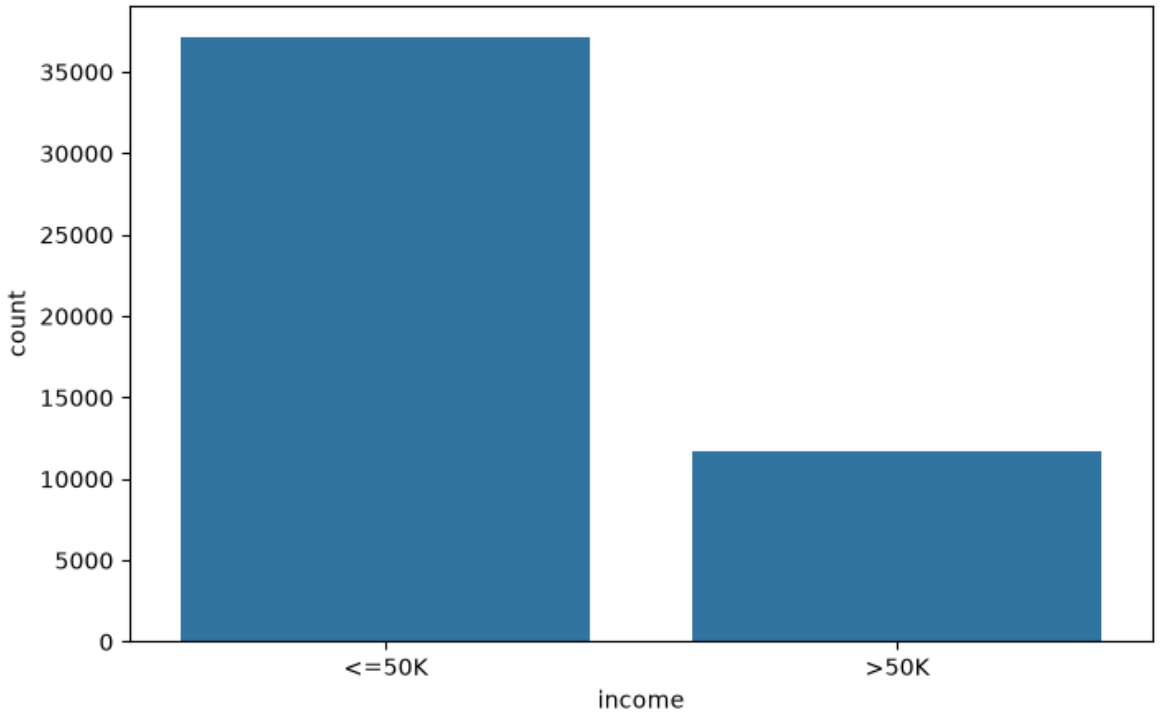
```
Portugal                        67
Taiwan                          65
Iran                            59
Nicaragua                       49
Greece                          49
Peru                            46
Ecuador                         45
France                          38
Ireland                         37
Thailand                        30
Hong                            30
Cambodia                        28
Trinadad&Tobago                 27
Laos                            23
Yugoslavia                      23
Outlying-US(Guam-USVI-etc)      23
Scotland                        21
Honduras                        20
Hungary                         19
Holand-Netherlands               1
Name: count, dtype: int64
-------------------
Attribute name: income
-------------------
income
<=50K    37155
>50K     11687
Name: count, dtype: int64
-------------------
```

In [6]:
```
# Import library seaborn untuk visualisasi
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

# Plot figure untuk menentukan distribusi kelas
plt.figure(figsize=(8,5))

# Menghitung baris setiap kelas
sns.countplot(x="income", data=data)
```

Out[6]:
```
<Axes: xlabel='income', ylabel='count'>
```



Diketahui bahwa dataset memiliki distribusi kelas yang tidak seimbang (imbalanced) sehingga secara teknis akan digunakan teknik untuk menangani data yang tidak seimbang.

Dataset preparation

In [7]:
```
# Buat salinan dataframe
df = data.copy(deep = True)

# Mengubah nilai "?" nilai ke bentuk Na / NaN untuk diproses lebih lanjut
for col in data.columns:
  df[[col]] = data[[col]].replace('?',np.nan)
```

In [8]:
```
# Melakukan seleksi kolom fitur/feature columns dari dataset
null_data = df.iloc[:,:-1]

# Temukan nilai null untuk semua atribut dan jumlahkan total nilai null
null_data.isnull().sum()
```

Out[8]:
```
age                  0
workclass         2799
fnlwgt               0
education            0
educational-num      0
marital-status       0
occupation        2809
relationship         0
race                 0
gender               0
capital-gain         0
capital-loss         0
hours-per-week       0
native-country     857
dtype: int64
```

In [9]:
```
# Hapus (drop) semua baris yang memiliki nilai null
df = df.dropna()

# Pilih kolom fitur (feature columns) dari dataset
null_data = df.iloc[:,:-1]

# Cek ulang nilai null
null_data.isnull().sum()
```

Out[9]:
```
age               0
workclass         0
fnlwgt            0
education         0
educational-num   0
marital-status    0
occupation        0
relationship      0
race              0
gender            0
capital-gain      0
capital-loss      0
hours-per-week    0
native-country    0
dtype: int64
```

StandardScaler *adalah class dari sklearn untuk melakukan normalisasi data agar data yang digunakan tidak memiliki penyimpangan yang besar.*

In [10]:
```
# Import library standard scaler
from sklearn.preprocessing import StandardScaler

# Buat dataframe dengan tipe data int64
colname= []
for col in df.columns:
  if pd.api.types.is_integer_dtype(df[col]):
      colname.append(col)

# Buat salinan dataset untuk keperluan persiapan data (data preparation)
df_copy = df.copy(deep = True)
df_fe = df.copy()

# Buat dataframe untuk fitur kategoris (categorical features)
df_fe.drop('income',axis='columns', inplace=True)
df_fe.drop(colname,axis='columns', inplace=True)

# Buat dataframe untuk kelas target (target class)
df_cl = df.copy()
df_cl.drop(df_copy.iloc[:,:-1],axis='columns', inplace=True)

# Membuat objek scaler (scaler object)
std_scaler = StandardScaler()
std_scaler

# Normalisasikan atribut numerik dan tetapkan ke dalam dataframe baru
df_norm = pd.DataFrame(std_scaler.fit_transform(df_copy[colname]), column
```

In [11]:
```
# Import library Ordinal Encoder dari package library sklearn.preprocessi
from sklearn.preprocessing import OrdinalEncoder
```

```
ord_enc = OrdinalEncoder()

# Encode fitur kategoris/categorical features menjadi fitur numerik (nume
for col in df_fe.columns[:]:
  if pd.api.types.is_string_dtype(df_fe[col]):
    df_fe[col] = ord_enc.fit_transform(df_fe[[col]])

# Encode label kategorikal (categorical label) menjadi label biner (binar
df_cl["income"] = np.where(df_cl["income"].str.contains(">50K"), 0, 1)
```

In [12]:
```
# Masukkan kolom id ke dataset yang berbeda
df_norm.insert(0, 'id', range(0, 0 + len(df_norm)))
df_fe.insert(0, 'id', range(0, 0 + len(df_fe)))
df_cl.insert(0, 'id', range(0, 0 + len(df_cl)))

# Lihat shapes dataset yang telah di proses
print(df_norm.shape)
print(df_fe.shape)
print(df_cl.shape)
```

```
(45222, 7)
(45222, 9)
(45222, 2)
```

In [13]:
```
# Gabungkan semua dataset
df_feature = pd.merge(df_norm,df_fe, on=["id"])
df_final = pd.merge(df_feature,df_cl, on=["id"])

# Drop kolom id dari gabungan dataset
df_final.drop('id',axis='columns', inplace=True)

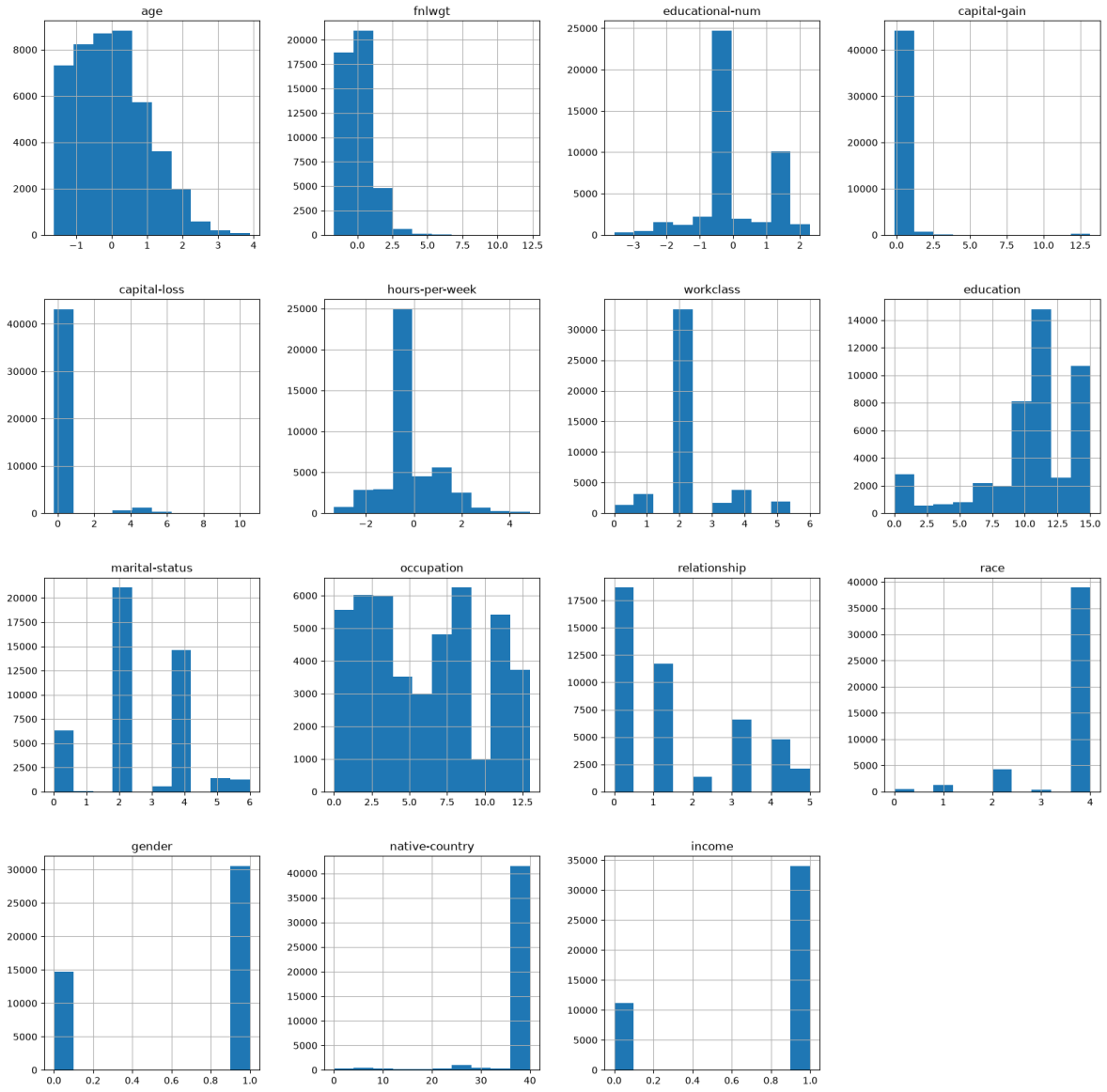
# Lihat 5 data teratas dari gabungan dataset
df_final.head(5)
```

Out[13]:

	age	fnlwgt	educational-num	capital-gain	capital-loss	hours-per-week	workclass	educ
0	-1.024983	0.350889	-1.221559	-0.146733	-0.21878	-0.078120	2.0	
1	-0.041455	-0.945878	-0.438122	-0.146733	-0.21878	0.754701	2.0	
2	-0.798015	1.393592	0.737034	-0.146733	-0.21878	-0.078120	1.0	
3	0.412481	-0.278420	-0.046403	0.877467	-0.21878	-0.078120	2.0	
4	-0.344079	0.084802	-1.613277	-0.146733	-0.21878	-0.910942	2.0	

Visualization

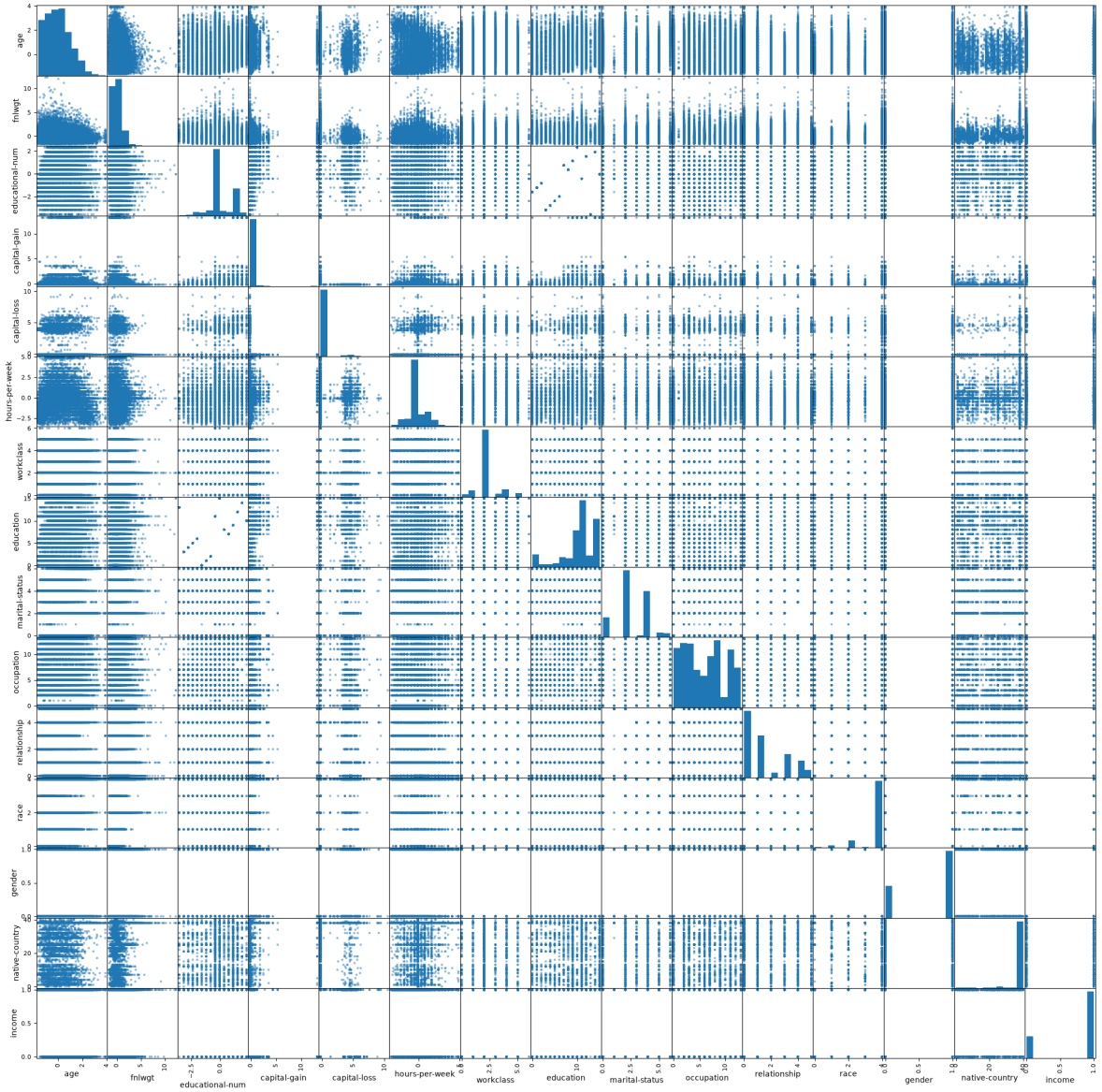
In [14]:
```
p = df_final.hist(figsize = (20,20))
```



Scatter matrix plot adalah plot yang digunakan untuk membuat sekumpulan scatter plot dari beberapa pasang variabel. Hal ini sangat bermanfaat terutama ketika ingin menganalisis bagaimana bentuk hubungan antar variabel. Plot ini sangat bermanfaat untuk digunakan untuk data yang ukurannya tidak terlalu besar. Untuk menggunakan scatter matrix kita harus memanggil fungsi *scatter_matrix* dari *pandas.plotting*

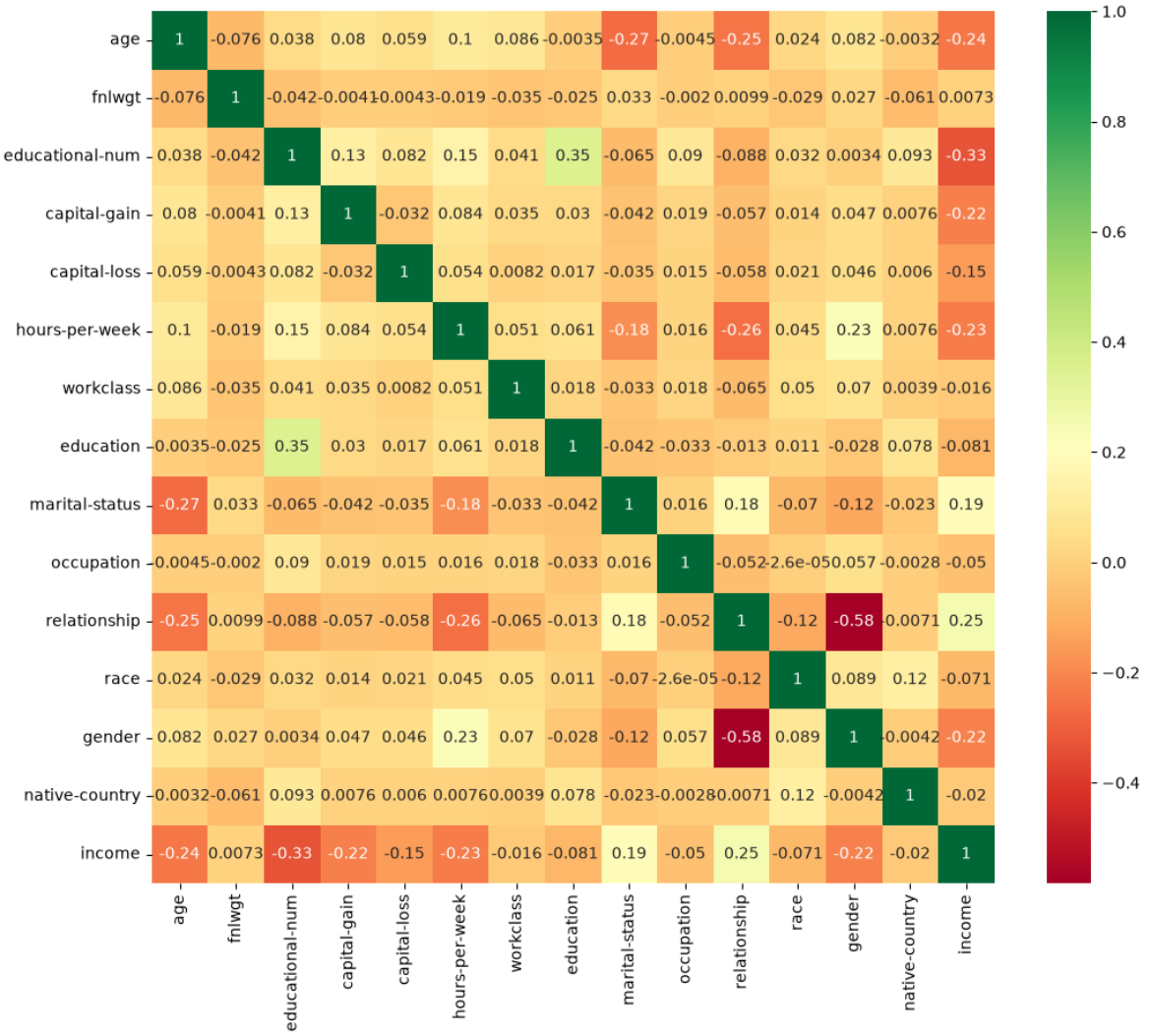
In [15]:
```
from pandas.plotting import scatter_matrix

p=scatter_matrix(df_final,figsize=(25, 25))
```



In [16]:
```
# Buat visualisasi korelasi data dengan heatmap
import seaborn as sns
import matplotlib.pyplot as plt

# Plot heatmap
plt.figure(figsize=(12,10))
p=sns.heatmap(df_final.corr(), annot=True,cmap ='RdYlGn')
```



Proses Modelling dengan KNN

In [17]:

```
import numpy as np
from sklearn.model_selection import KFold

# Kolom terakhir adalah label (income), jadi hanya kolom sebelumnya yang
X=df_final.iloc[:,:-1].to_numpy()
y=df_final.iloc[:,-1:].to_numpy()
kf = KFold(n_splits=5)
kf.get_n_splits(X)

print(kf)

for train_index, test_index in kf.split(X):
    print("TRAIN:", train_index, "TEST:", test_index)
    x_train, x_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
```

```
KFold(n_splits=5, random_state=None, shuffle=False)
TRAIN: [ 9045  9046  9047 ... 45219 45220 45221] TEST: [   0    1    2 ...
9042 9043 9044]
TRAIN: [    0     1     2 ... 45219 45220 45221] TEST: [ 9045  9046  9047
... 18087 18088 18089]
TRAIN: [    0     1     2 ... 45219 45220 45221] TEST: [18090 18091 18092
... 27131 27132 27133]
TRAIN: [    0     1     2 ... 45219 45220 45221] TEST: [27134 27135 27136
... 36175 36176 36177]
TRAIN: [    0     1     2 ... 36175 36176 36177] TEST: [36178 36179 36180
... 45219 45220 45221]
```

In [18]:

```
print('-------- x axis test ----------')
print(x_test)
print('-------- x axis train ---------')
print(x_train)
print('-------- y axis test ----------')
print(y_test)
print('-------- y axis train ---------')
print(y_train)

print('******************************')
```

```
-------- x axis test ----------
[[ 0.63944887  1.45787715 -0.43812161 ...  2.          1.
  38.        ]
 [ 1.24469679 -1.49349376 -0.046403   ...  4.          1.
  38.        ]
 [-0.26842301 -0.14603391 -0.046403   ...  4.          1.
  38.        ]
 ...
 [ 1.47166476 -0.35805983 -0.43812161 ...  4.          0.
  38.        ]
 [-1.25195088  0.11127873 -0.43812161 ...  4.          1.
  38.        ]
 [ 1.01772882  0.92951628 -0.43812161 ...  4.          0.
  38.        ]]
-------- x axis train ---------
[[-1.02498291  0.35088942 -1.22155881 ...  2.          1.
  38.        ]
 [-0.04145504 -0.94587846 -0.43812161 ...  4.          1.
  38.        ]
 [-0.79801494  1.39359159  0.73703421 ...  4.          1.
  38.        ]
 ...
 [-0.344079   -0.77568407 -0.43812161 ...  4.          1.
  10.        ]
 [ 0.86641684  0.04524191  1.52047141 ...  2.          1.
  37.        ]
 [-0.64670296  0.84029651 -0.43812161 ...  2.          0.
  38.        ]]
-------- y axis test ----------
[[1]
 [1]
 [1]
 ...
 [1]
 [1]
 [0]]
-------- y axis train ---------
[[1]
 [1]
 [0]
 ...
 [0]
 [1]
 [1]]
******************************
```

In [19]:

```
# Import library model KNN dengan alias 'KNeighborsClassifier'
from sklearn.neighbors import KNeighborsClassifier
import sklearn.metrics as metrics
import matplotlib.pyplot as plt

# Buat variabel kosong untuk menyimpan metrik KNN/KNN metrics
scores=[]

# Kita coba nilai k yang berbeda untuk KNN (dari k=1 sampai k=20)
lrange=list(range(1,20))

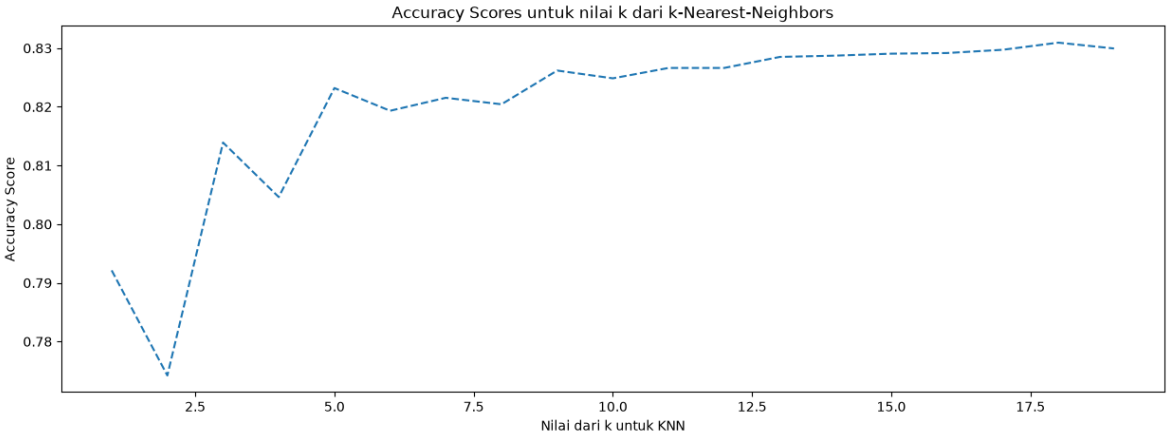
# Proses perulangan untuk KNN
for k in lrange:
  knn=KNeighborsClassifier(n_neighbors=k) # Masukkan nilai k dan ukuran '
```

```
  knn.fit(x_train,y_train.ravel()) # Masukan data train/data latih untuk
  y_pred=knn.predict(x_test) # Lihat prediksi KNN dengan memasukkan data
  scores.append(metrics.accuracy_score(y_test,y_pred)) # Tambahkan perfor
plt.figure(2,figsize=(15,5))

optimal_k = lrange[scores.index(max(scores))]
print("Nilai k KNN yang optimal adalah %d" % optimal_k)
print("Skor optimalnya adalah %.2f" % max(scores))

# Visualisasikan hasilnya
plt.plot(lrange, scores,ls='dashed')
plt.xlabel('Nilai dari k untuk KNN')
plt.ylabel('Accuracy Score')
plt.title('Accuracy Scores untuk nilai k dari k-Nearest-Neighbors')
plt.show()
```

```
Nilai k KNN yang optimal adalah 18
Skor optimalnya adalah 0.83
```



Tugas Analisis

Berdasarkan hasil percobaan KNN di atas, buatlah hasil analisis dengan menjawab pertanyaan berikut:

1. Pada hasil diagram heatmap, terlihat banyak konfigurasi warna dan nilai yang ditampilkan. Jelaskan arti kolom berwarna merah pada hasil diagram heatmap yang dihasilkan! Pasangan variabel apa saja yang berwarna merah?
2. Apa yang terjadi jika nilai test size diganti menjadi 0.3 kemudian 0.2 dan 0.1?
3. Buatlah tabel yang berisi hasil pengujian Anda dan sertakan beberapa nilai k dan nilai akurasi untuk masing-masing skenario uji!
4. Apakah terjadi perbedaan nilai akurasi model? Jika ya, jelaskan pendapat Anda mengapa hal tersebut bisa terjadi.
5. Berapa nilai presisi dan recall dari masing-masing model yang Anda coba, serta jelaskan arti dari nilai tersebut.

In [20]:
```
# No 1
# Heatmap memakai cmap 'RdYlGn': MERAH = korelasi negatif, HIJAU = korela
corr = df_final.corr()
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
pairs = corr.where(mask).stack().reset_index()
pairs.columns = ['variabel_1', 'variabel_2', 'korelasi']
```

```python
# Ambil semua pasangan dengan korelasi negatif, urutkan dari yang paling
negatif = pairs[pairs['korelasi'] < 0].sort_values('korelasi')
print("Semua pasangan dengan korelasi negatif (semakin kecil = semakin me
display(negatif.reset_index(drop=True))

print("\n5 pasangan paling negatif (paling merah):")
display(negatif.head(5).reset_index(drop=True))
```

```
Semua pasangan dengan korelasi negatif (semakin kecil = semakin merah):
```

	variabel_1	variabel_2	korelasi
0	relationship	gender	-0.582974
1	educational-num	income	-0.332800
2	age	marital-status	-0.271623
3	hours-per-week	relationship	-0.259194
4	age	relationship	-0.247564
5	age	income	-0.237040
6	hours-per-week	income	-0.227199
7	capital-gain	income	-0.221034
8	gender	income	-0.215760
9	hours-per-week	marital-status	-0.183178
10	capital-loss	income	-0.148687
11	marital-status	gender	-0.120008
12	relationship	race	-0.118584
13	educational-num	relationship	-0.088431
14	education	income	-0.081196
15	age	fnlwgt	-0.075792
16	race	income	-0.070844
17	marital-status	race	-0.070302
18	workclass	relationship	-0.065049
19	educational-num	marital-status	-0.064986
20	fnlwgt	native-country	-0.061098
21	capital-loss	relationship	-0.058425
22	capital-gain	relationship	-0.056818
23	occupation	relationship	-0.051644
24	occupation	income	-0.049787
25	capital-gain	marital-status	-0.042399
26	fnlwgt	educational-num	-0.041993
27	education	marital-status	-0.041520
28	fnlwgt	workclass	-0.034970
29	capital-loss	marital-status	-0.034820
30	education	occupation	-0.033499
31	workclass	marital-status	-0.033221
32	capital-gain	capital-loss	-0.032102

	variabel_1	variabel_2	korelasi
33	fnlwgt	race	-0.029188
34	education	gender	-0.027569
35	fnlwgt	education	-0.025318
36	marital-status	native-country	-0.022721
37	native-country	income	-0.020103
38	fnlwgt	hours-per-week	-0.018679
39	workclass	income	-0.015659
40	education	relationship	-0.013179
41	relationship	native-country	-0.007144
42	age	occupation	-0.004474
43	fnlwgt	capital-loss	-0.004349
44	gender	native-country	-0.004170
45	fnlwgt	capital-gain	-0.004110
46	age	education	-0.003511
47	age	native-country	-0.003180
48	occupation	native-country	-0.002848
49	fnlwgt	occupation	-0.002036
50	occupation	race	-0.000026

`5 pasangan paling negatif (paling merah):`

	variabel_1	variabel_2	korelasi
0	relationship	gender	-0.582974
1	educational-num	income	-0.332800
2	age	marital-status	-0.271623
3	hours-per-week	relationship	-0.259194
4	age	relationship	-0.247564

Jawaban No 1

Heatmap dibuat dengan `cmap='RdYlGn'`, sehingga skala warnanya: **merah = korelasi negatif (mendekati -1)**, kuning = mendekati 0 (tidak ada hubungan linear), **hijau = korelasi positif (mendekati +1)**. Diagonal berwarna hijau tua karena setiap variabel berkorelasi 1 dengan dirinya sendiri.

Jadi sel **merah** berarti dua variabel memiliki hubungan linear **berlawanan arah**: ketika nilai satu variabel naik, nilai variabel lainnya cenderung turun. Semakin merah pekat, semakin kuat hubungan negatifnya.

Pasangan variabel dengan warna merah paling kuat adalah:

- **relationship – gender = -0.583**. Hubungannya negatif karena `gender` sudah di-encode menjadi angka (Female = 0, Male = 1), sedangkan kategori `relationship` juga di-encode berurutan. Nilai kategorinya saling berlawanan (misalnya `Husband` vs `Wife`), sehingga korelasinya negatif cukup kuat.
- **educational-num – income = -0.333**. Semakin tinggi pendidikan, kecenderungannya semakin besar untuk masuk kelas `>50K` (income = 0), sehingga korelasinya negatif.

Pasangan negatif lainnya memiliki korelasi lemah (mendekati 0), sehingga warnanya hanya sedikit merah/kekuningan, misalnya `age – marital-status` (-0.272), `hours-per-week – relationship` (-0.259), dan `age – income` (-0.237).

In [21]:
```
# No 2
# Membagi data dengan test_size 0.3, 0.2, dan 0.1 (fitur saja, label inco
from sklearn.model_selection import train_test_split

X_feat = df_final.iloc[:, :-1].to_numpy()
y_all = df_final.iloc[:, -1].to_numpy()

for ts in [0.3, 0.2, 0.1]:
    xtr, xte, ytr, yte = train_test_split(X_feat, y_all, test_size=ts, ra
    print(f"test_size={ts}: train={len(xtr)} ({1-ts:.0%}), test={len(xte)
```

```
test_size=0.3: train=31655 (70%), test=13567 (30%)
test_size=0.2: train=36177 (80%), test=9045 (20%)
test_size=0.1: train=40699 (90%), test=4523 (10%)
```

Jawaban No 2

Mengubah `test_size` hanya mengubah **proporsi pembagian data**:

- **0.3** : 70% data latih, 30% data uji
- **0.2** : 80% data latih, 20% data uji
- **0.1** : 90% data latih, 10% data uji

Semakin kecil `test_size`, data latih makin banyak sehingga KNN punya lebih banyak tetangga referensi, tetapi data uji makin sedikit sehingga estimasi akurasi makin kurang stabil. Dampak nyata pada akurasi dapat dilihat di tabel No 3.

> Catatan: pada sel KFold di atas, `X` sudah dibuat dari `df_final.iloc[:, :-1]` sehingga kolom label `income` **tidak** ikut menjadi fitur. Jika label ikut menjadi fitur, akurasi akan melonjak karena model "melihat jawabannya" (data leakage); perbandingannya ada di sel No 4.

In [22]:
```
# No 3
# Tabel hasil pengujian: beberapa nilai k x tiga skenario test_size
import sklearn.metrics as metrics
from sklearn.neighbors import KNeighborsClassifier

k_list = [1, 3, 5, 7, 9, 11, 15, 19]
test_sizes = [0.3, 0.2, 0.1]
```

```
splits = {}
rows = []
for ts in test_sizes:
    xtr, xte, ytr, yte = train_test_split(X_feat, y_all, test_size=ts, ra
    splits[ts] = (xtr, xte, ytr, yte)
    for k in k_list:
        knn = KNeighborsClassifier(n_neighbors=k).fit(xtr, ytr)
        rows.append({'test_size': ts, 'k': k, 'akurasi': metrics.accuracy

hasil = pd.DataFrame(rows)
tabel = hasil.pivot(index='k', columns='test_size', values='akurasi')
tabel.columns = [f"test_size={c}" for c in tabel.columns]
display(tabel.style.format("{:.4f}").highlight_max(axis=0, color='lightgr

print("\nk terbaik per skenario:")
for ts in test_sizes:
    sub = hasil[hasil.test_size == ts]
    best = sub.loc[sub.akurasi.idxmax()]
    print(f"  test_size={ts}: k={int(best.k)}, akurasi={best.akurasi:.4f}
```

k	test_size=0.1	test_size=0.2	test_size=0.3
1	0.7886	0.7882	0.7889
3	0.8163	0.8168	0.8160
5	0.8236	0.8247	0.8206
7	0.8293	0.8263	0.8253
9	0.8282	0.8286	0.8270
11	0.8353	0.8310	0.8286
15	0.8355	0.8331	0.8286
19	0.8337	0.8317	0.8291

```
k terbaik per skenario:
  test_size=0.3: k=19, akurasi=0.8291
  test_size=0.2: k=15, akurasi=0.8331
  test_size=0.1: k=15, akurasi=0.8355
```

Jawaban No 3

Tabel di atas berisi akurasi KNN untuk beberapa nilai k (1, 3, 5, 7, 9, 11, 15, 19) pada tiga skenario `test_size` (0.3, 0.2, 0.1). Sel hijau menandai akurasi tertinggi di setiap skenario.

k	test_size=0.3	test_size=0.2	test_size=0.1
1	0.7889	0.7882	0.7886
3	0.8160	0.8168	0.8163
5	0.8206	0.8247	0.8236
7	0.8253	0.8263	0.8293
9	0.8270	0.8286	0.8282

k	test_size=0.3	test_size=0.2	test_size=0.1
11	0.8286	0.8310	0.8353
15	0.8286	**0.8331**	**0.8355**
19	**0.8291**	0.8317	0.8337

Nilai k terbaik pada masing-masing skenario:

- `test_size=0.3` → **k = 19**, akurasi **0.8291**
- `test_size=0.2` → **k = 15**, akurasi **0.8331**
- `test_size=0.1` → **k = 15**, akurasi **0.8355**

In [23]:
```
# No 4
# Selisih akurasi antar skenario + perbandingan dengan versi yang bocor (
ringkas = tabel.agg(['min', 'max', 'mean'])
ringkas.loc['selisih (max-min)'] = ringkas.loc['max'] - ringkas.loc['min'
display(ringkas.style.format("{:.4f}"))
print("Selisih rata-rata akurasi antar skenario:",
      f"{(tabel.mean().max() - tabel.mean().min()):.4f}")

# Perbandingan leakage: X berisi kolom income (seperti sel KFold modul)
X_leak = df_final.to_numpy()
xtr, xte, ytr, yte = train_test_split(X_leak, y_all, test_size=0.2, rando
acc_leak = metrics.accuracy_score(yte, KNeighborsClassifier(n_neighbors=5
acc_ok = tabel.loc[5, 'test_size=0.2']
print(f"\nAkurasi k=5, test_size=0.2 -> tanpa leakage: {acc_ok:.4f} | den
```

	test_size=0.1	test_size=0.2	test_size=0.3
min	0.7886	0.7882	0.7889
max	0.8355	0.8331	0.8291
mean	0.8238	0.8225	0.8205
selisih (max-min)	0.0469	0.0449	0.0402

```
Selisih rata-rata akurasi antar skenario: 0.0033
Akurasi k=5, test_size=0.2 -> tanpa leakage: 0.8247 | dengan leakage: 0.94
52
```

Jawaban No 4

Ya, nilai akurasi **sedikit berbeda** antar skenario. Dari tabel ringkasan, selisih akurasi maksimum–minimum di tiap skenario adalah 0.0402–0.0469, sedangkan selisih rata-rata akurasi antar skenario hanya **0.0033** (0.8355 vs 0.8291 pada k terbaik). Penyebabnya:

1. **Jumlah data latih berbeda.** Semakin kecil `test_size`, KNN memiliki lebih banyak data referensi sehingga pencarian tetangga terdekat cenderung lebih representatif dan akurasi cenderung sedikit naik.
2. **Komposisi data uji berbeda.** Data uji yang lebih kecil (misalnya 10%) membuat akurasi lebih sensitif terhadap beberapa sampel sulit, sehingga hasilnya lebih fluktuatif dan kurang stabil dibanding data uji 30%.
3. **Pengaruh nilai k.** k kecil (mis. 1) sensitif terhadap noise dan cenderung overfitting (akurasi hanya sekitar 0.789); k yang lebih besar lebih halus. Nilai k terbaik bisa

bergeser antar skenario.

4. **Kelas tidak seimbang.** Mayoritas data berlabel `<=50K`, sehingga akurasi mudah terlihat tinggi walau model lemah pada kelas minoritas (lihat precision/recall di No 5).

Perbandingan di sel atas juga menunjukkan bahwa jika kolom label `income` ikut menjadi fitur, akurasi melonjak dari **0.8247** menjadi **0.9452** (pada `test_size=0.2`, k=5). Inilah yang disebut **data leakage**: model seolah-olah "melihat jawabannya". Karena itu hasil yang valid adalah hasil yang fiturnya tidak menyertakan label.

In [24]:
```
# No 5
# Precision & recall tiap model (k terbaik per skenario) + k=5 sebagai pe
from sklearn.metrics import precision_score, recall_score, classification

rows = []
for ts in test_sizes:
    xtr, xte, ytr, yte = splits[ts]
    sub = hasil[hasil.test_size == ts]
    k_best = int(sub.loc[sub.akurasi.idxmax(), 'k'])
    for k in sorted({5, k_best}):
        yp = KNeighborsClassifier(n_neighbors=k).fit(xtr, ytr).predict(xt
        rows.append({
            'test_size': ts, 'k': k,
            'akurasi': metrics.accuracy_score(yte, yp),
            'precision (<=50K)': precision_score(yte, yp, pos_label=1),
            'recall (<=50K)': recall_score(yte, yp, pos_label=1),
            'precision (>50K)': precision_score(yte, yp, pos_label=0),
            'recall (>50K)': recall_score(yte, yp, pos_label=0),
        })
pr = pd.DataFrame(rows)
display(pr.style.format({c: "{:.4f}" for c in pr.columns if c not in ('te

# Detail lengkap untuk satu model
xtr, xte, ytr, yte = splits[0.2]
yp = KNeighborsClassifier(n_neighbors=5).fit(xtr, ytr).predict(xte)
print("\nClassification report (test_size=0.2, k=5)  [0 = >50K, 1 = <=50K
print(classification_report(yte, yp, target_names=['>50K', '<=50K']))
```

	test_size	k	akurasi	precision (<=50K)	recall (<=50K)	precision (>50K)	recall (>50K)
0	0.300000	5	0.8206	0.8646	0.9029	0.6595	0.5708
1	0.300000	19	0.8291	0.8620	0.9201	0.6952	0.5529
2	0.200000	5	0.8247	0.8688	0.9033	0.6663	0.5861
3	0.200000	15	0.8331	0.8680	0.9175	0.6974	0.5767
4	0.100000	5	0.8236	0.8676	0.9033	0.6646	0.5816
5	0.100000	15	0.8355	0.8686	0.9206	0.7056	0.5772

```
Classification report (test_size=0.2, k=5)  [0 = >50K, 1 = <=50K]
              precision    recall  f1-score   support

       >50K       0.67      0.59      0.62      2242
      <=50K       0.87      0.90      0.89      6803

    accuracy                           0.82      9045
   macro avg       0.77      0.74      0.75      9045
weighted avg       0.82      0.82      0.82      9045
```

Jawaban No 5

Nilai precision dan recall tiap model ada pada tabel di atas (dihitung untuk kedua kelas). Pada encoding modul ini, label **`0 = >50K`** dan **`1 = <=50K`**.

Ringkasan nilai yang diperoleh:

test_size	k	akurasi	precision <=50K	recall <=50K	precision >50K	recall >50K
0.3	5	0.8206	0.8646	0.9029	0.6595	0.5708
0.3	19	0.8291	0.8620	0.9201	0.6952	0.5529
0.2	5	0.8247	0.8688	0.9033	0.6663	0.5861
0.2	15	0.8331	0.8680	0.9175	0.6974	0.5767
0.1	5	0.8236	0.8676	0.9033	0.6646	0.5816
0.1	15	0.8355	0.8686	0.9206	0.7056	0.5772

Arti nilai tersebut:

- **Precision** = TP / (TP + FP): dari semua data yang diprediksi sebagai suatu kelas, berapa persen yang benar. Precision `>50K` yang tinggi berarti jika model bilang seseorang berpenghasilan `>50K`, prediksinya jarang salah.
- **Recall** = TP / (TP + FN): dari semua data yang sebenarnya anggota suatu kelas, berapa persen yang berhasil ditemukan model. Recall `>50K` yang rendah berarti banyak orang berpenghasilan `>50K` yang terlewat (diprediksi `<=50K`).

Karena kelas `<=50K` jauh lebih banyak, precision dan recall kelas ini tinggi (precision sekitar 0.86–0.87 dan recall sekitar 0.90–0.92), sedangkan kelas minoritas `>50K` lebih rendah (precision sekitar 0.66–0.71 dan recall sekitar 0.55–0.59). Model dengan akurasi tertinggi (`test_size=0.1`, k=15) belum tentu terbaik untuk kelas `>50K`: recall-nya 0.5772, sedikit di bawah model `test_size=0.2`, k=5 yang recall-nya 0.5861. Inilah alasan akurasi saja tidak cukup untuk menilai model pada data imbalanced.

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/